

Advanced Reading in Computer Vision (MAT3563)

LAB 01 – FEATURE EXTRACTION AND ITS APPLICATION

A. PHƯƠNG PHÁP TÌM GÓC HARRIS

Trong phần thực hành này chúng ta sẽ thực hiện quá trình tìm góc theo thuật toán Harris Conner Detect. Các bước của thuật toán này như sau:

- 1) Tính các đạo hàm cấp 2 của ảnh theo mỗi hướng I_{xx} , I_{yy} , I_{xy}
- 2) Tính

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

- 3) Tính

$$R = \det M - k (\text{trace } M)^2$$

Với

$$\begin{aligned} \det M &= \lambda_1 \lambda_2 \\ \text{trace } M &= \lambda_1 + \lambda_2 \end{aligned}$$

- 4) Chuẩn hóa giá trị của R về đoạn [0, 1].
- 5) Lấy ngưỡng để lọc R – Thay đổi các giá trị ngưỡng để được số điểm góc hợp lý, các điểm được coi là điểm đáng quan tâm nếu:
 - a. $R \geq \text{threshold}$;
 - b. (x, y) là điểm cực đại cục bộ (local maxima) của R: $R(x, y) \geq R(x', y')$ với mọi (x', y') trong số 8 láng giềng của (x, y) .

Trong bài giảng chúng ta có thể lấy $w(x, y)$ là hàm cửa sổ - tức là $w(x, y) = 1$ nếu (x, y) thuộc cửa sổ đang xét và 0 nếu ngược lại; hoặc là lấy dạng lọc Gaussian.

Chương trình nguồn đã có trong tệp đính kèm Eg1_Harris_Conner_Detect_numpy_cv.ipynb. Các bạn có thể thử với các ảnh tương ứng hoặc các ảnh mới.

1. Xây dựng code dựa theo công thức của phương pháp

Trước hết chúng ta sẽ tự xây dựng code dựa theo các bước và công thức tương ứng trong lý thuyết.

- Trong đoạn code thứ nhất của tệp Eg1_Harris_Conner_Detect_numpy_cv.ipynb, chúng ta xây dựng các ma trận lọc (filter/mask) gồm Sobel_x, Sobel_y và Gaussian. Chúng ta có thể thay thế Gaussian bằng hàm cửa sổ bất kỳ.
- Đoạn code thứ 2 xây dựng phép tính tích chập 2 chiều (convolution 2D). Ở đây sau khi tính tích chập chúng ta sẽ chuẩn hóa bằng cách đưa kết quả về kiểu nguyên với giá trị thuộc [0, 255] (ứng với cường độ sáng).
- Đoạn lệnh thứ 3 là phương thức harris(img, threshold=0.6, k = 0.05): Trong đó threshold là ngưỡng và $k \in (0, 1)$ là tham số tính R.

Các bước tính toán trong phương thức harris theo đúng các công thức của phương pháp được trình bày ở trên.

```
def harris(img, threshold=0.6, k = 0.05):  
  
    img_cpy = img.copy() # copying image  
  
    img1_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # grayscaling (0-1)  
  
    dx = convolve(img1_gray, SOBEL_X) # convolving with sobel filter on X-axis  
    dy = convolve(img1_gray, SOBEL_Y) # convolving with sobel filter on Y-axis  
    # square of derivatives  
    dx2 = np.square(dx)  
    dy2 = np.square(dy)  
    dxdy = dx*dy #cross filtering  
    # gauss filter for all directions (x,y,cross axis)  
    g_dx2 = convolve(dx2, GAUSS)  
    g_dy2 = convolve(dy2, GAUSS)  
    g_dxdy = convolve(dxdy, GAUSS)  
    # Harris Function # R = (harris) = det - k*(trace**2)  
    harris = g_dx2*g_dy2 - np.square(g_dxdy) - k*np.square(g_dx2 + g_dy2)  
    # Normalizing inside (0-1)  
    cv2.normalize(harris, harris, 0, 1, cv2.NORM_MINMAX)  
  
    # find all points above threshold (nonmax supression line)  
    loc = np.where(harris >= threshold)
```

```
# drawing filtered points
for pt in zip(*loc[::-1]):
    cv2.circle(img_cpy, pt, 3, (255, 0, 0), -1)

return img_cpy, g_dx2, g_dy2, dx, dy, loc
```

Ngoài đoạn code trên, chúng ta cũng có thể tham khảo đoạn code cho phương pháp này, Eg2_Harris_Conner_Detect_numpy_cv.ipynb, trong đó sử dụng hàm của thư viện OpenCV cho phần lọc Gaussian; hoặc code sử dụng cả phần tính Gradients (tích chập với bộ lọc Sobel) và Gaussian đều dùng của thư viện OpenCV, như trong hai phần code tiếp theo của tệp chương trình.

2. Sử dụng thư viện OpenCV.

Trong phần code ví dụ này chúng ta sẽ sử dụng thư viện OpenCV thực hiện việc tìm điểm góc theo thuật toán Harris. Trong thư viện OpenCV, chúng ta có thể sử dụng phương thức: `cv2.cornerHarris(img, blockSize, kernelSize, k)` cho mục đích này. Các đối số của nó là:

`img` - Hình ảnh đầu vào, nó phải có thang độ xám và kiểu float32.

`blockSize` - Đó là kích thước của vùng lân cận được xem xét để phát hiện góc

`kernelSize` - Tham số kích thước mặt nạ Sobel được sử dụng.

`k` - Tham số tự do của bộ dò Harris trong phương trình.

Đoạn code sau đây thực hiện việc sử dụng thư viện trên để tìm các điểm góc:

```
import cv2
import numpy as np

filename = 'D:\\t1.png'
img = cv2.imread(filename)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

gray = np.float32(gray)
dst = cv2.cornerHarris(gray, 2, 3, 0.04)

#result is dilated for marking the corners, not important
dst = cv2.dilate(dst, None)

# Threshold for an optimal value, it may vary depending on the image.
img[dst>0.01*dst.max()]=[0,0,255]

cv2.imshow('dst',img)
if cv2.waitKey(0) & 0xff == 27:
    cv2.destroyAllWindows()
```

Bài tập tự thực hành

- So sánh code trong Eg1_Harris_Conner_Detect_numpy_cv và Eg2_Harris_Conner_Detect_numpy_cv (đính kèm) với thuật toán để tìm ra đoạn code nào đầy đủ hơn. Hãy sửa file code bị thiếu cho đúng thuật toán.
- Trong các chương trình trên, hãy sử dụng hàm cửa sổ dạng $w(x, y) = \{0 \mid 1\}$ thay vì sử dụng của sổ Gaussian như trong phương pháp gốc. Quan sát sự thay đổi của kết quả trên một ảnh bất kỳ.
- Sử dụng ảnh test2.png được giáo viên cung cấp
- Hãy sử dụng cùng một tham số của thuật toán Harris Conner Detect cho cả 2 nửa ảnh để tìm các điểm góc trên mỗi nửa ảnh
- Chú ý mỗi nửa ảnh có cùng chiều cao và chiều ngang = $\frac{1}{2}$ ảnh chung. Hãy vẽ đường nối các điểm góc có tọa độ gần nhau nhất trong ảnh.
- Sử dụng ảnh t1.png đính kèm và tạo ảnh đối chứng bằng cách quay ảnh gốc một góc bất kỳ, nhưng giữ nguyên kích thước phần **nội dung** ảnh. Sau đó áp dụng phương pháp Harris để tìm các điểm đặc trưng trên hai ảnh. So sánh vị trí của chúng.

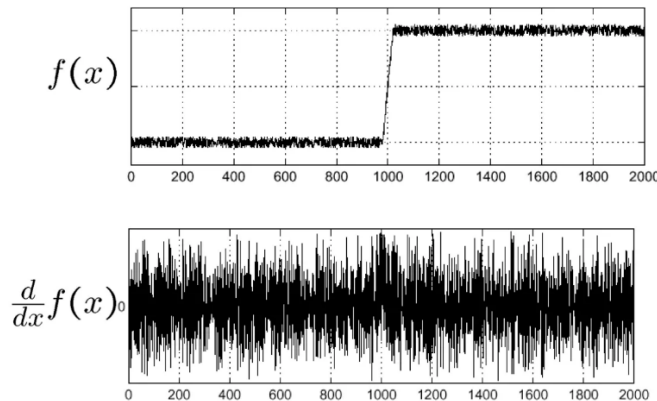
B. SCALE INVARIANT FEATURE TRANSFORMATION: HARRIS - LAPLACIAN

Tiếp theo, chúng ta sử dụng phương pháp Harris – Laplacian để xây dựng phương pháp cho phép tìm các điểm đặc trưng có tính bất biến với phép co giãn (scale).

Trong thuật toán Harris, để tránh nhiễu, đôi khi ta nhân chập ảnh ban đầu với một filter Gaussian (ví dụ lọc trung bình) ứng với một Gaussian $G(\sigma_D) \sim N(0, \sigma_D)$:

$$M(\sigma_I, \sigma_D) = \boxed{g(\sigma_I)} * \begin{bmatrix} I_x^2(\sigma_D) & I_x I_y(\sigma_D) \\ I_x I_y(\sigma_D) & I_y^2(\sigma_D) \end{bmatrix}$$

Trong đó $I(\sigma_D) = I * G(\sigma_D)$. Việc này là do phép lấy đạo hàm rất nhạy cảm với nhiễu, như hình dưới:



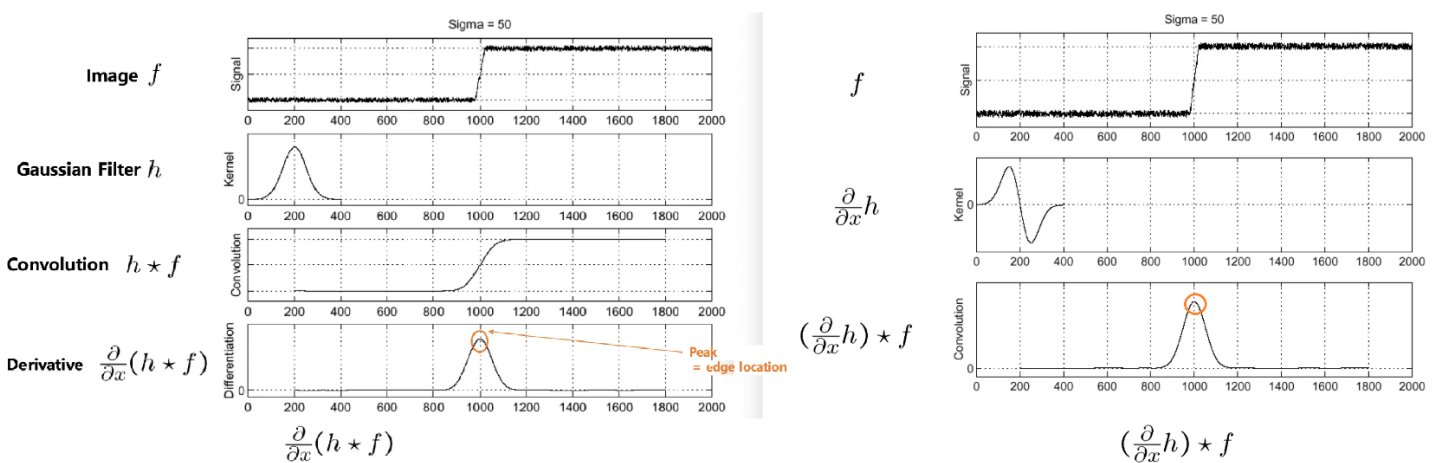
Tính nhạy cảm với nhiễu của phép lấy đạo hàm

(i) Derivative of Gaussian

Trong phương pháp Harris và các phương pháp phát triển từ ý tưởng này như Harris – Laplacian of Gaussian hoặc Difference of Gaussian, chúng ta sử dụng cách tính toán dựa trên tính chất kết hợp của phép lấy đạo hàm và phép nhân chập. Giả sử f là ảnh đầu vào và h là mặt nạ nhân chập, ta có:

$$\frac{\partial}{\partial x}(h * f) = \left(\frac{\partial}{\partial x}h\right) * f$$

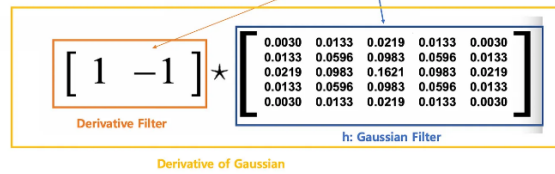
Hình dưới cho thấy kết quả tương đồng giữa 2 phương pháp tính toán áp dụng với bộ lọc h = Gaussian:



Từ đây, thay cho tính tích chập sau đó lấy đạo hàm trên kết quả, ta có thể tạo ra bộ lọc mới bằng cách áp dụng phép lấy đạo hàm trên chính bộ lọc h , sau đó nhân chập bộ lọc mới với ảnh f .

Trong các thuật toán dưới đây sử dụng tích chập với lọc Gauss nên cách tính toán trên còn gọi là đạo hàm của Gauss (Derivative of Gaussian). Cách tính toán được minh họa trong hình dưới, ở đây ta xét đạo hàm theo hướng x:

$$\frac{\partial}{\partial x}(h \star f) = (\frac{\partial}{\partial x}h) \star f$$



(ii) Laplacian

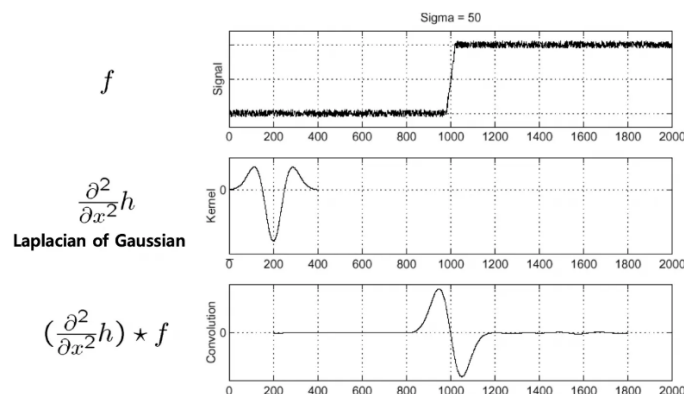
Đây chính là tổng hợp đạo hàm riêng cấp 2 theo từng hướng x và y. Áp dụng trên lưới điểm rời rạc (ảnh số) ta có thể sử dụng công thức:

$$\begin{aligned} \text{Laplacian } \nabla^2 f &= \frac{\partial^2 f}{\partial^2 x} + \frac{\partial^2 f}{\partial^2 y} \\ &= f(x+1, y) + f(x-1, y) - 2f(x, y) + f(x, y+1) + f(x, y-1) - 2f(x, y) \\ &= f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y) \end{aligned}$$

Do vậy có thể biểu diễn nó như là bộ lọc tích chập (Ta có thể sử dụng các mặt nạ tích chập với nhiều hướng hơn, ví dụ 8 hướng => phần tử trung tâm là 8)

0	-1	0
-1	4	-1
0	-1	0

Tương tự trường hợp đạo hàm cấp 1, ta cũng có cách tính toán gọi là Laplacian of Gaussian, tức là áp toán tử Laplacian lên bộ lọc Gaussian để thu được bộ lọc mới, sau đó mới áp dụng lên ảnh gốc. Kỹ thuật tạo bộ lọc mới này gọi là Laplacian of Gaussian (**LoG**):



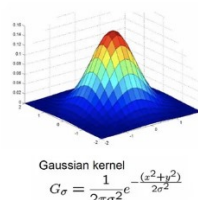
Hình dưới minh họa việc tính mặt nạ mới LoG, với toán tử Laplacian được lấy như ma trận ở trên:

$$LoG = -\frac{1}{\pi\sigma^4} \left[1 - \frac{x^2 + y^2}{2\sigma^2} \right] e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

0	-1	0
-1	4	-1
0	-1	0

Laplacian Filter

★



Gaussian Filter

0	1	1	2	2	2	1	1	0
1	2	4	5	5	5	4	2	1
1	4	5	3	0	3	5	4	1
2	5	3	-12	-24	-12	3	5	2
2	5	0	-24	-40	-24	0	5	2
2	5	3	-12	-24	-12	3	5	2
1	4	5	3	0	3	5	4	1
1	2	4	5	5	5	4	2	1
0	1	1	2	2	2	1	1	0

LoG Filter

1. PHƯƠNG PHÁP HARRIS – LAPLACIAN of GAUSSIAN

Phương pháp Harris – Laplacian có 02 thay đổi chính so với phương pháp Harris gốc:

- 1) Thay cho hàm cửa sổ $w(x, y)$ (có thể dùng hàm Gaussian $g(\sigma_i)$ trong công thức trên) như ở trong phương pháp gốc, chúng ta dùng kernel là Laplacian of Gaussian:

$$LoG = \sigma^2 (G_{xx}(x, y, \sigma) + G_{yy}(x, y, \sigma))$$

Ở đây G_{xx} , G_{yy} là đạo hàm riêng cấp 2 của G (là kết quả sau lọc gauss: $G = I * G(\sigma_D)$), theo hướng x và y :

$$G_{xx} = \frac{\partial^2 G}{\partial x^2}; \quad G_{yy} = \frac{\partial^2 G}{\partial y^2}$$

- 2) Sử dụng một dãy các σ_n khác nhau (trong bài báo gốc, tác giả sử dụng tất cả 19 giá trị σ_1 đến σ_{19})

Convolution with Gaussian -> Convolution with LoG

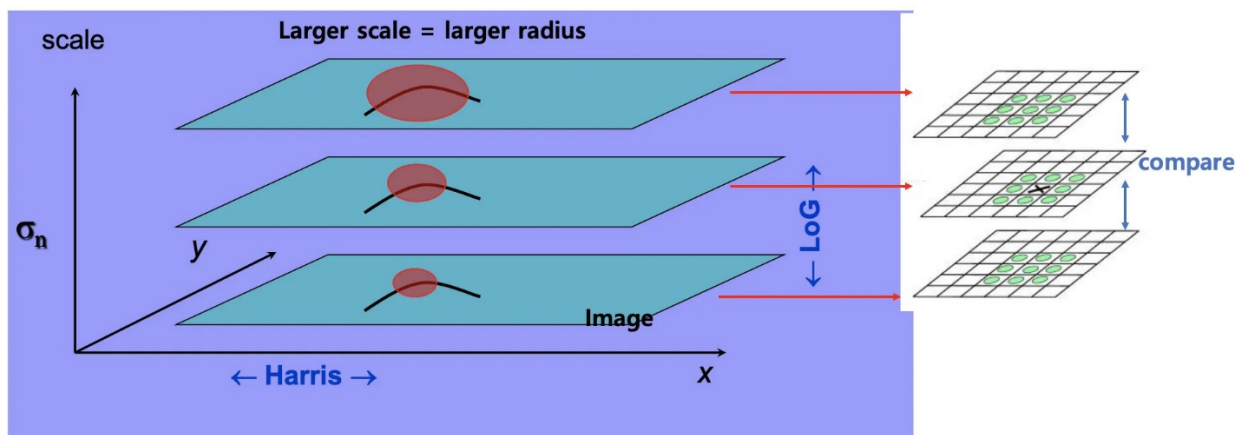
Original second-moment matrix M and Harris response function R :

$$M(\sigma_n, \sigma_D) = \boxed{g(\sigma_n)} * \begin{bmatrix} I_x^2(\sigma_D) & I_x I_y(\sigma_D) \\ I_x I_y(\sigma_D) & I_y^2(\sigma_D) \end{bmatrix}$$

↑
Replaced by $LoG(\sigma_n)$

$$R = \det[M(\sigma_n, \sigma_D)] - \alpha [\text{trace}(M(\sigma_n, \sigma_D))]^2 = g(I_x^2)g(I_y^2) - [g(I_x I_y)]^2 - \alpha [g(I_x^2) + g(I_y^2)]^2$$

- 3) Bằng việc này, kết quả thu được là dữ liệu 3 chiều (3D), trong đó 2D gốc là ứng với ảnh; chiều thứ 3 ứng với các σ_n khác nhau



- 4) Các điểm góc (cực đại của ma trận R thu được theo phương pháp Harris), bây giờ sẽ được lấy cực đại cả theo các σ_n - một cách cục bộ (tức là so sánh theo các σ_n liên tiếp).

2. CÁC VẤN ĐỀ CẦN XỬ LÝ

- Thuật toán Harris (có thể dùng $G(\sigma_D)$ hoặc không) tương tự với phần A.
- Trong thuật toán Harris, chúng ta cần thay kernel là hàm cửa sổ $w(x, y)$ bởi kernel dạng *Laplacian of Gaussian* như trong công thức trên. Với mỗi σ_n ta có thể tính được kernel theo công thức này nếu biết mặt nạ (ma trận) Gaussian tương ứng.
- Tuy nhiên để xây dựng LoG đúng với tư tưởng của phương pháp, kích thước của ma trận Gaussian nói trên sẽ thay đổi tùy theo σ_n (tạo thành các scale khác nhau).

Trước hết chúng ta cần tìm hiểu chi tiết về cách tính toán với LoG cho ảnh đầu vào.

Chúng ta có thể lần lượt giải quyết các vấn đề trên bằng các đoạn code dưới đây:

- (i) Khởi dựng ma trận Laplacian size 3 x 3:

```
laplacianKernel = np.array([[ -1,  -1,  -1],
                             [ -1,  8,  -1],
                             [ -1,  -1,  -1]], np.int)
```

- (ii) Khởi dựng ma trận Laplacian of Gaussian 5 x 5 – Với mức σ_1 cơ sở ta sẽ sử dụng ma trận này

```
log5Kernel = np.array([[0, 0, 1, 0, 0],
                        [0, 1, 2, 1, 0],
                        [1, 2, -16, 2, 1],
                        [0, 1, 2, 1, 0],
                        [0, 0, 1, 0, 0]], np.int)
```

(iii) Tạo ma trận ứng với σ_n từ ma trận cơ sở - Ở đây ta cần tính lại kích thước của kernel dựa vào σ_n

```
def gaussianGenerator(self, size = 0, sigma = 0):
    if size == 0 and sigma != 0:
        # Calculate kernel size for gaussian blur base on sigma
        size = 2 * math.ceil(3 * sigma) + 1

    if size != 0 and sigma == 0:
        # Calculate sigma base on kernel size
        sigma = (size // 2) / 2

    # Separate gaussian into 2 direction: vertical & horizontal
    # Especially, both vertical & horizontal filter are symmetric
    # GaussX
    gaussX = gaussianXFunction(size, sigma)
    # GaussY is a transpose matrix of gaussX because they are symmetric
    gaussY = gaussX.transpose()

    # Normalize gaussian kernel by averaging
    sumX = np.sum(gaussX)
    sumY = np.sum(gaussY)

    gaussX = gaussX/sumX
    gaussY = gaussY/sumY

    self.gaussKernelX = gaussX
    self.gaussKernelY = gaussY

    return gaussX, gaussY
```

Ở đây chúng ta sử dụng phương thức tạo ma trận theo hướng X (sau đó chuyển vị để lấy theo hướng Y do tính chất đối xứng của chúng. Phương thức tạo Gaussian theo hướng X:

```
def gaussianXFunction(size, sigma):
    minX = -size/2
    maxX = size/2
    step = 1

    # a, b
    a_range = np.arange(minX, maxX - step + 1.0, step)
    b_range = np.arange(minX + step, maxX + 1.0, step)

    # Vectorize gaussX_int function
    vec_gaussX_int = np.vectorize(gaussX_int)

    # Replicate gaussX_int function for each a, b
    gaussX = vec_gaussX_int(integrand, sigma, a_range, b_range)

    return gaussX.reshape(-1, 1)
```

(iv) Tính giá trị LoG ứng với σ_n cụ thể

```
def LoG_function(sigma):
    ksize = 2 * math.ceil(3 * sigma) + 1
    # Kernel size - look back gaussianGenerator

    radius = ksize // 2
    # Generate orthogonal vertical and horizontal grid
```

```

x, y = np.ogrid[-radius:radius+1, -radius:radius+1]

# Need to normalize LoG by multiply LoG function with sigma^2 to reduce the
effect of sigma
# Because When increase sigma, the response of blob will decrease.
# Note: Need a little bit transform LoG function to get good result
ex = np.exp(-(x**2) / (2.0*sigma*sigma))
ey = np.exp(-(y**2) / (2.0*sigma*sigma))

# LoG filter (blob filter)
# Note: x.shape (2*radius + 1, 1), y.shape (1, 2*radius + 1)
# So it use broadcast in numpy to compute (x*x + y*y) and ex*ey
LoG = (-2*sigma**2 + (x*x + y*y)) * (ex*ey) * (1.0 / (2*np.pi*sigma**4))

return LoG

```

- (v) Tính toán các điểm đặc trưng (điểm góc theo Harris) với công thức LoG thay cho cửa sổ $w(x, y)$, ứng với các level cụ thể của σ_n

```

def detectByLoG(logKernel, img):
    # With Declared Convolve() method - Convolve image with LoG filter
    logImg = convolve(img, logKernel)
    return logImg

```

- (vi) Xây dựng đầu ra 3D theo các giá trị σ_n khác nhau:

```

def create_log_pyramid(self, img):
    # Build scale-space pyramid
    pyramid = []
    # Define lambda expression to compute next_sigma
    next_sigma = lambda k, sigma: k*sigma

    # Declare myfilter obj
    myfilter = []

    for i in range(self.no_scale_lv):
        # Compute current sigma
        curSigma = next_sigma(CONSTANT_K**i, self.initSigma)
        # Generate LoG filter
        myfilter = LoG_function(curSigma)
        # Convolve image with that filter
        logImg = detectByLoG(img, myfilter)
        # Padding logImg with constant value 0 to find extrema
        logImg = np.pad(logImg, (1, 1), mode = 'constant')
        # Squaring the response of blob to make response strong positive
        # to high contrast
        logImg = np.square(logImg)

        # Store in pyramid
        pyramid.append(logImg)

    # Store pyramid in class with numpy type
    return np.array(pyramid)

```

- (vii) Tiếp theo, chúng ta sẽ sử dụng dữ liệu đầu ra dạng 3D nói trên, so sánh điểm góc (theo Harris) ứng với các level σ_n khác nhau, từ đó tìm ra điểm cực đại theo level của σ_n . Như lý thuyết, đây mới là các điểm đặc trưng sẽ bất biến với phép co giãn và chúng ta sẽ giữ lại.

```

initSigma = 1.0 # (basis  $\sigma_1 = 1$ )
no_scale_lv = 9

def detectBlobByLoG(img):
    # Create scale-space pyramid with 9 LoG images for 9 level of  $\sigma_n$ 

```



```

LoG_pyramid = create_log_pyramid(img)
# Find maxima of squared Laplacian response in scale-space
keypoints = []

# Get size of original image
iH, iW = img.shape

# # MAIN IDEA:
# # Traverse through each image to find maximum
# for x in range(1, iH - 1):
#     for y in range(1, iW - 1):
#         # Take a region around each pixel
#         # of each LoG image with 9*3*3 all scale to compare
#         roiScale = self.LoG_pyramid[:, x - 1: x + 1 + 1, y-1:y+1+1]
#         # Find maximum point
#         max = np.amax(roiScale)

#         # Check high contrast
#         if max >= CONTRAST_THRESHOLD:
#             # Find index of maximum point in roiScale
#             # iz: iz-th layer of scale-space
#             iz, ix, iy = np.unravel_index(roiScale.argmax(), roiScale.shape)
#             # Push to keypoints list
#             # Store location and scale of this keypoint
#             keypoints.append((x + ix - 1, y + iy - 1,
#                               CONSTANT_K*iz*self.initSigma))

# # Eliminate some duplicate keypoint because
# # we get max of 9*3*3 neighbors at each time
# self.keypoints = list(set(keypoints))

# Pre-Define indices of neighbors of pixel[i][j] to speed up when computing
xidx = np.array([-1, -1, -1, 0, 0, 1, 1, 1, 0])
#Vertical axe: relative neighbor for x coordinate
yidx = np.array([-1, 0, 1, -1, 1, -1, 0, 1, 0])
#Horizontal axe: relative neighbor for y coordinate

# Traverse through scale-space
# Skip the topmost and lowermost.
# Because they dont have enough neighbor to compare
for iz in range(1, no_scale_lv - 1):
    cur_img = LoG_pyramid[iz]
    prev_img = LoG_pyramid[iz - 1]
    next_img = LoG_pyramid[iz + 1]
    # Traverse LoG image to find local maxima
    for x in range(1, iH):
        for y in range(1, iW):
            # Check maxima
            '''
            One pixel in an image is compared with its 8 neighbours as well as 9
pixels
            in next scale and 9 pixels in previous scales.
            '''
            if (np.all(cur_img[x][y] >= cur_img[x + xidx[:-1], y + yidx[:-1]]) \
                and np.all(cur_img[x][y] >= prev_img[x + xidx, y + yidx]) \
                and np.all(cur_img[x][y] >= next_img[x + xidx, y + yidx])):
                '''
                * Getting rid of low-contrast and edge keypoints
                * Rejection:
                - Low contrast: Threshold intensities ( $|D(x)| < 0.03$  in term of
range[0, 1]) then it is rejected.
                - Lie on edge: Use concept similar with harris

```



```

        but In SIFT, efficiency is increased by just calculating the
        ratio of these two eigenvalues.
        
$$\text{Tr}(H)^2 / \text{Det}(H) < (r+1)^2 / r$$
 , where  $r = 10$ 
        Which eliminates keypoints that have a ratio between the
        principal curvatures greater than 10.
        * Therefore, the remains is strong keypoints.
        '''

    # Check if it is low contrast
    if cur_img[x][y] < CONTRAST_THRESHOLD:
        continue

    # Check edge
    # 2x2 Hessian matrix (H) computed at the location and
    # scale of the keypoint:
    # Because i use x as vertical and y as horizontal,
    # when computing derivative of x and y need to
    # revert x, y syntax
    dxx = cur_img[x][y - 1] + cur_img[x][y + 1] - 2*cur_img[x][y]
    dyy = cur_img[x - 1][y] + cur_img[x + 1][y] - 2*cur_img[x][y]
    dxy = (cur_img[x - 1][y - 1] + cur_img[x + 1][y + 1] \
           - cur_img[x - 1][y + 1] \
           - cur_img[x + 1][y - 1]) / 4.0

    # Compute trace and det of H
    trH = dxx + dyy
    detH = dxx*dyy - dxy**2

    # Compute curvature_ratio
    curvature_ratio = np.nan_to_num(trH**2 / detH)

    if curvature_ratio <= self.ratio_threshold:
        # Store location and scale of this keypoint
        keypoints.append((x - 1, y - 1, \
                          CONSTANT_K**iz*self.initSigma))

# Store keypoints to class
return keypoints

```

- (viii) Sau khi có các điểm đặc trưng từ thuật toán Harris - Laplacian of Gaussian, ta có thể vẽ các điểm đó lên ảnh bằng phương thức dưới đây

```

def plotBlob(img):
    fig, axes = plt.subplots()

    # Determine Set keypointtr of input img
    keypoints = detectBlobByLoG(img)

    # Set title and show img
    axes.set_title("Blob detector used Laplacian of Gaussian")
    axes.imshow(img, interpolation='nearest', cmap="gray")

    for blob in keypoints:
        # Get location and sigma of blob
        x, y, sigma = blob
        # Note: radius of blob = sigma * sqrt(2)
        radius = sigma * CONSTANT_K
        circle = plt.Circle((y, x), radius, color='red', linewidth=1.5, fill=False)
        axes.add_patch(circle) # Add circle to axis

    # Turn off axis
    axes.set_axis_off()

```

Đến đây chúng ta có thể ghép các đoạn code này, kết hợp với phần A để có chương trình đầy đủ.

3. BÀI TẬP TỰ THỰC HÀNH

- Kết hợp các đoạn code để có chương trình đầy đủ tìm điểm đặc trưng theo thuật toán Harris Laplacian of Gaussian.
- Sử dụng ảnh book1.png và book2.png được giáo viên cung cấp
 - o Tìm điểm góc trên mỗi ảnh
 - o Hãy vẽ đường nối các điểm góc có liên quan với nhau nhất trong 2 ảnh.
- Sử dụng ảnh t1.png đính kèm và tạo ảnh đối chứng bằng cách quay ảnh gốc một góc bất kỳ, và giãn ảnh theo tỷ lệ 1:2. Sau đó áp dụng phương pháp Harris Laplacian of Gaussian để tìm các điểm đặc trưng trên hai ảnh. So sánh vị trí của chúng.
- Trong đoạn code (vii) – Ta đang sử dụng cách tính toán tương tự cách tính SIFT (DoG) thay vì Harris, tức là thay vì xét giá trị $R = \det(M) - k \cdot [\text{Trace}(M)]^2$ như trong phần A, chúng ta sử dụng $r := [\text{Trace}(M)]^2 / \det(M)$. Các vị trí được coi là điểm đặc trưng (key-point) cần thỏa mãn:
 $r < (R+1)^2 / R$ – trong đó R là ngưỡng chọn (CURVATURE_THRESHOLD) và thường lấy $R = 10$.
Hãy thay đổi đoạn code này trở về sử dụng phương pháp Harris để xác định góc.

C. SCALE INVARIANT FEATURE: DIFFERENCES of GAUSSIAN (DoG)

Phương pháp Laplacian of Gaussian (LoG) – Harris xác định được các điểm đặc trưng bất biến với phép quay và phép co giãn. Tuy nhiên việc tính toán LoG vẫn khá phức tạp. Chúng ta có thể xấp xỉ Laplacian of Gaussian dựa vào tính chất dưới đây. Trước hết, ta viết lại công thức của LoG, đồng thời áp dụng lấy đạo hàm cho G, ta có 2 công thức dưới đây:

$$\text{LoG} = \sigma^2 \nabla^2 G \quad \text{và} \quad \frac{\partial G}{\partial \sigma} = \sigma \nabla^2 G$$

Với k – số thực bất kỳ, ta có thể xấp xỉ $\frac{\partial G}{\partial \sigma}$ trong công thức trên bởi:

$$\frac{\partial G}{\partial \sigma} \approx \frac{G(x, y, k\sigma) - G(x, y, \sigma)}{k\sigma - \sigma}$$

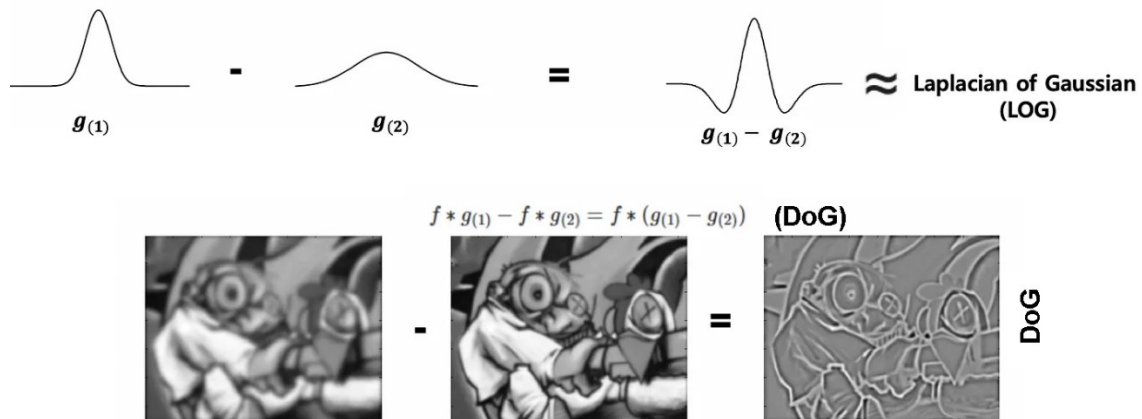
Và do đó suy ra

$$\frac{G(x, y, k\sigma) - G(x, y, \sigma)}{k\sigma - \sigma} \approx \sigma \nabla^2 G$$

Do vậy:

$$G(x, y, k\sigma) - G(x, y, \sigma) \approx (k - 1)\sigma^2 \nabla^2 G = (k - 1) \text{LoG}$$

Trong phần tiếp theo ta sẽ thay LoG bởi xấp xỉ của nó, thu được bằng việc trừ các Gaussian với phương sai khác nhau (Difference of Gaussian - DoG). Hình dưới cho thấy sự xấp xỉ của LoG và DoG



Ngoài tính DoG với các phương sai khác nhau, chúng ta còn xây dựng lược đồ tính toán kiểu Pyramid, tức là chia các nhóm giá trị của phương sai thành các Octaves, trong đó tại Octaves thứ k , phương sai sẽ là 2^k lần phương sai cơ sở (tại Octaves thứ 0). Điều này dẫn tới kích thước các mặt nạ Gauss trong tính toán cũng sẽ thay đổi.

DIFFERENCE OF GAUSSIAN (DOG)

Công thức tính toán với DoG có thể tham khảo từ slide lý thuyết. Các vấn đề cần thực hiện khi triển khai:

- Xây dựng hệ thống Pyramid Octaves và phương sai tương ứng
- Tính r (như phần B) – CURVATURE_RATE, tức là thay vì xét giá trị $R = \det(M) - k \cdot [\text{Trace}(M)]^2$ như trong phần A, chúng ta sử dụng $r := [\text{Trace}(M)]^2 / \det(M)$. Các vị trí được coi là điểm đặc trưng (key-point) cần thỏa mãn:
 $r < (R+1)^2 / R$ – trong đó R là ngưỡng chọn (CURVATURE_THRESHOLD) và thường lấy $R = 10$.
- Tính toán dữ liệu dạng 3D kết quả của các DoG
- Tìm cực đại cục bộ theo cả 3 chiều của dữ liệu 3D trên

Bài tập: Tham khảo tệp chương trình `dog_harris_detections.ipynb` sử dụng thuật toán DoG để tìm điểm quan trọng (key-point) từ một bức ảnh. Hãy đọc kỹ chương trình và phân tích tác dụng, mục đích của mỗi đoạn chương trình trong đó, so sánh với các phần trong thuật toán lý thuyết tương ứng. Viết một báo cáo ngắn trình bày những phân tích trên.

D. KẾT HỢP DIFFERENCES of GAUSSIAN (DoG) VÀ ÁP DỤNG VÀO GHÉP ẢNH

Tiếp theo chúng ta sẽ sử dụng thuật toán RANSAC để thực hiện việc ghép ảnh (alignment) thành panorama, dựa vào tập điểm key-point đã tìm theo phương pháp DoG ở trên. Lý thuyết về thuật toán RANSAC (ghép các tập key-point trong 02 ảnh) các bạn có thể đọc thêm từ slide. Chương trình nguồn các bạn có thể tham khảo từ file đính kèm `dog_harris_alignment_customH_fixed.ipynb` cho trường hợp chỉ có 02 ảnh, hoặc `multistitch_customH_multiband.ipynb` cho trường hợp ghép nhiều ảnh.

Dưới đây là một số giải thích, mô tả từng đoạn trong chương trình nguồn ghép 02 ảnh.

1. Cấu hình và thư viện

```
import cv2, numpy as np, matplotlib.pyplot as plt
```

```
from pathlib import Path
```

- **cv2**: OpenCV – dùng cho xử lý ảnh, phát hiện đặc trưng, homography, warp ảnh.
- **numpy**: Xử lý ma trận, toán học số.
- **matplotlib**: Hiển thị ảnh trong notebook.
- **Path**: Quản lý đường dẫn thư mục một cách gọn gàng.

Thư mục `OUT_DIR` được tạo ra để lưu tất cả kết quả (ảnh keypoints, matches, panorama...).

2. Phát hiện điểm đặc trưng (DoG + Harris)

Gaussian pyramid & DoG

- **Gaussian pyramid**: mờ ảnh ở nhiều mức sigma để có ảnh đa tỉ lệ.
- **DoG pyramid**: lấy hiệu giữa 2 ảnh Gaussian liên tiếp → phát hiện điểm cực trị (local extrema) theo không gian–tỉ lệ (scale-space).

```
build_gaussian_pyramid(gray, num_octaves, s, sigma0)
```

```
build_dog_pyramid(gauss_pyr)
```

→ Trả về các mức ảnh mờ và DoG tương ứng.

Kiểm tra cực trị & loại bỏ cạnh

```
is_local_extrema(cube)
```

```
pass_edge_response(dog, y, x, r)
```

- **is_local_extrema**: so sánh điểm hiện tại với lân cận trong không gian 3D (trước–sau trong pyramid).
- **pass_edge_response**: loại bỏ điểm nằm trên cạnh (edge-like response) dựa vào ma trận Hessian (theo ý tưởng SIFT).

Harris corner filter

```
harris_response(img, k, win_sigma)
```

- Tính ma trận cấu trúc (second-moment matrix) và đáp ứng Harris $R = \det(M) - k \cdot \text{trace}^2(M)$.

- Giữ lại các điểm vừa là cực trị DoG vừa có **R lớn** → chính là điểm góc mạnh, ổn định.

Non-max suppression

`nonmax_suppression(points, radius)`

- Loại bỏ keypoint lân cận yếu, giữ lại điểm mạnh nhất trong bán kính NMS.

Kết quả: một danh sách keypoints (tọa độ, octave, sigma, score).

3. Chuyển keypoints sang dạng OpenCV + mô tả đặc trưng

`kps_to_cv2_keypoints(kps)`

`compute_orb_descriptors(gray, kps)`

- Chuyển danh sách $\{x, y, \sigma, \dots\}$ thành `cv2.KeyPoint` để OpenCV có thể dùng.
- Tính **descriptor ORB** tại các keypoints này.

→ ORB là mô tả đặc trưng nhị phân, nhanh và dễ match bằng Hamming distance.

4. Ghép điểm đặc trưng giữa ảnh

`match_descriptors(desc1, desc2, ratio=0.75)`

- Dùng **BFMatcher** (Brute Force Matcher) với khoảng cách Hamming.
- **Ratio test (Lowe's test)**: chỉ giữ match m tốt nhất nếu $\text{dist}(m) < 0.75 \cdot \text{dist}(n)$ (loại match không chắc chắn).

5. Ước lượng Homography

`estimate_homography(kps1_cv, kps2_cv, matches, ransac_thresh=3.0)`

- Lấy tọa độ keypoints từ cặp match.
- Dùng **cv2.findHomography** với **RANSAC** để tính ma trận 3×3 **H** biến đổi ảnh 2 sang hệ quy chiếu ảnh 1.
- Mask inliers cho biết match nào “đúng” theo homography.

6. Warp ảnh và ghép

Warp ảnh

`cv2.warpPerspective(img2, T @ H, (pano_w, pano_h))`

- Ánh xạ toàn bộ ảnh 2 sang canvas chung (theo H và dịch T để tránh tọa độ âm).
- Tính kích thước canvas bằng cách project các góc ảnh.

Blend ảnh

Có hai phiên bản:

1. **Feather blending** (bản trước): tính khoảng cách đến rìa vùng overlap rồi pha trộn tuyến tính theo trọng số.

2. Multi-band blending (bản sau):

- Xây **Gaussian & Laplacian pyramid** cho mỗi ảnh.
- Xây pyramid cho mask.
- Trộn ở từng tầng (low + high frequency).
- Dựng lại ảnh → seam mượt hơn.

7. Stitch nhiều ảnh

- Tính keypoints và homography của tất cả ảnh so với ảnh đầu tiên.
- Warp từng ảnh lên canvas chung.
- Blend lần lượt → panorama cuối cùng.

8. Xuất kết quả

- `cv2.imwrite(...)`: lưu overlay keypoints, matches, warped images, panorama.
- `plt.imshow(...)`: hiển thị trực quan trong notebook.

Bài tập:

- 1) Hãy thực nghiệm các đoạn chương trình trên và bổ sung phần đánh dấu các điểm key-point tương ứng với nhau (được nối bằng đường minh họa liên kết) trên 2 ảnh đang cần ghép.
- 2) Viết một báo cáo ngắn về thuật toán Ransac được sử dụng trong các phương pháp ghép ảnh đã ví dụ ở trên.